Array:

In JS, an array is an orderd list of values. Each value, known as element assigned with numeric position in the array called its index.

⇒ The indexing start at zero 0, so first element is at 0 index.

⇒ Array can hold any type of data; numbers, strings, object even an array of

Declaration and Intialization

[] is used to declare and intialize array.

```
let a = [];  // Empty Array
let b = [10, 20, 30];
let c = new Array(1, 2, 3);
```

using array constructor

Indexing

Array indexing refers to process of accesing individual elements with in an array.

zero-indexing:

In javascript, array uses 0-based indexing, means position of first element start from zero index.

Accessing Element:

To access an element, the array is followed by square bracket [] containing the index of the desired value.

Assignment:

Array Indexing also used to modify the value of an element at specific index.

Array Length

We can get length of array using array length property.

```
let a = ["HTML", "CSS", "JS"];
let len = a.length;
console.log("Array Length " + len);
```

Increase length of Array:

```
let a = ["HTML", "CSS", "JS"];
a.length = 7;
console.log(a);
['HTML', 'CSS', 'JS', '', '', '', ''];
```

Decrease length of array:

```
a.length = 2;
console.log(a);
['HTML', 'CSS']
```

Array methods

toString()

The toString() method converts given value into the string with each element seperated by comma

```
let a = ["HTML", "CSS", "JS", "React"];
let s = a.toString();
console.log(s);
```

Output: HTML, CSS, JS, React

join()

This join method creates and returns a new string by concatenating all elements of an array. It uses a specified separator b/w each element in the resulting string.

```
let a = ["HTML", "CSS", "JS"];
console.log(a.join('|'));
```

Output: HTML|CSS|JS

delete

The delete operator is used to delete the given value

```
let a= [ "HTML","CSS","JS"];
console.log( delete a[1]);
console.log(a);
```

Output:

true

HTML, null, JS

concat()

The concat() method is used to concatenate two or more arrays and it gives the merged array.

```
let a1 = [11, 12, 13];
let a2 = [14, 15, 16];
let a3 = [17, 18, 19];
let newArr = a1.concat(a2, a3);
console.log(newArr);
```

Output: [11, 12, 13, 14, 15, 16, 17, 18, 19]

flat()

The flat() method is used flatten the array i.e it merges all the given array and reduces all the nesting present in it.

```
const a1 = [['1','2'],['3',
'4','5'
```

push()

The push() method is used to add an element at the end of array.

```
let a =[10, 20, 30]
a.push(60);
  console.log(a)
[10, 20, 30, 60]
```

unshift()

to add an at start of array

```
let a = [ 20, 30, 40];
a.unshift( 10, 20);
console.log (a)
```

Output: [10, 20, 20, 30, 40]

pop()

pop() method is used to remove elements from end of array.

```
let a = [20, 30, 40, 50];
a.pop( );
console.log (a)
```

Output: [20, 30, 40]

shift()

The shift() method is to remove element from begining.

```
let a = [10, 20, 30];
    a.shif();
    console.log(a);
```

Output: [20, 30]

splice()

used to insert and remove element in b/w array

splice (x, y, z)

- x → index
- y → no. of removal element
- z → element going to be add

let

let a = [20, 30, 40]

a.splice(1, 2); → from index 1 remove 2 elements

Output: [20];

a.splice(1, 0, 3, 4, 5); → from index 1 remove no element, add 3, 4, 5 in array

console.log(a)

Output: [20, 3, 4, 5];

slice()

The slice method returns a new array containing a portion of original array, based on start

and end index provided as argument.

```
const a = [1, 2, 3, 4, 5, 6, 7]
const res = a.slice(1, 4);
```

inclusive index ↓ exclusive index ↓

output:

[2, 3, 4]

[1, 2, 3, 4, 5]

IndexOf():

used to find the index of particular element

```
let a = ["Hello", "World"];
console.log(a.indexOf('World');
```

Output: 1

includes()

To check wether the array contains specified element or not

```
let a = ["Hello", "World"];

console.log(a.includes("Hello");
console.log(a.includes("Bye");
```

Output:

true

false

sort()

The sort method sorts elements of an array in alphabetical order in ascending order.

```
let a = ["Hey", "I", "am", "Shumaila"];
```

find() , findIndex()

The find() method finds the first value which passes the user specified condition and findIndex() method finds out the first index value which passes the user-specified conditions.

```
let a = [1, 2, 3, 4, 5, 6, 7, 8]
let find = a.find (function
(element){
        return element > 4
});
console.log (find);
```

```
let val = a.findIndex(function
(element){
    return element >= 4
});
console.log(val);
```

Output:

5 → element

4 → index

Reverse

Used to reverse the order of elements

```
let a = [1, 2, 3, 4, 5];
a.reverse();
```

Output: [5, 4, 3, 2, 1]

Mutable methods:

Change the original array

⇒ push(), pop(), splice(), shif()

returns new array without changing the original.

⇒ slice(), filter(), map()

map()

⇒ transform values

⇒ runs a function on every elemen

⇒ returns a new array

```
Let nums = [1, 2, 3, 4];
let squares = nums.map
        (n => n*);
console.log (squares)
```

Output: [1, 4, 9, 16]

filter()

⇒ select values

⇒ run a condition on each element

⇒ returns a new array only matching value

```
let num = [10, 20, 30, 40]
let above20 = num.filter
        (n => n > 20);
console.log (above20);
```

Output => [30, 40]

reduce()

array.reduce (callbackFunction (accumulator, currentValue, currentIndex, array), initialValue)

accumulator: also known as total or previous value. It holds the intial value in begining then last returned value of callbackFn

currentValue: The value of current element being process

```
const nums= [1, 2, 3, 4, 5];
const sum = numbreduce(
(accumlator, currentValue) =>
{
return accumulator + currentVal;
0);
console.log (sum);
```

Output: 15